

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	CSA1220	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Constraint Based Problem Solving
Weightage:	40%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	Moniga V	Reg. No.:	192521184
Section / Batch:	2025 - 2029	Date:	26/08/2026

Instructions to Students

- Identify the relevant concepts of I/O interfaces, interrupts, DMA, bus arbitration, and high-speed communication protocols.
- Analyze the I/O requirements considering CPU utilization, latency, throughput, bandwidth, and device priorities.
- Apply suitable I/O techniques such as DMA, vectored interrupts, memory-mapped I/O, nested interrupts, and bus arbitration.
- Compute performance measures such as data transfer rate, interrupt latency, CPU utilization, throughput, and transfer time.
- Interpret the results by evaluating CPU efficiency, communication performance, interrupt response, and suitability of the proposed I/O architecture.

QUESTIONS

1. Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2. Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3. Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4. Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5. Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μ s
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct

I Moniga V (192521184) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Moniga V
RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------

Marks Evaluation:

Q. No	Problem Understanding (25)	Constraint Analysis (25)	Solution Development (25)	Application of Concepts (15)	Justification (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/25	/ 25	/ 15	/ 10	/ 100

Course Coordinator

Faculty Incharge

Question 1: I/O Communication Mechanism for a Real-Time Embedded System

Constraints Given

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

1. Problem Understanding

The system must service several sensors that each raise an interrupt once every 1 ms (an interrupt rate of 1 kHz per sensor), while keeping overall CPU utilization under 15%. Because response time is critical (real-time system) and multiple devices are involved, the interrupt-handling scheme must identify the interrupting device quickly (vectored interrupts) rather than polling, and the register-access method (memory-mapped vs I/O-mapped) must be chosen to minimize per-access latency.

2. Constraint Analysis

- Timing constraint: with a 1 ms inter-arrival time per sensor, the Interrupt Service Routine (ISR) execution time directly determines CPU load — long ISRs quickly violate the 15% ceiling.

- Interrupt-identification constraint: with multiple sensors, a polled scheme would force the CPU to check each device's status flag sequentially, wasting cycles. Vectored interrupts remove this overhead by letting each device supply an index/vector that directly points the CPU to its ISR address.
- Addressing-mode constraint: Memory-mapped I/O reuses the normal load/store instruction set and address bus, allowing any addressing mode and instruction (including compiler-generated code) to access device registers. I/O-mapped I/O needs dedicated IN/OUT instructions and a separate address space, which typically adds decode and instruction-fetch overhead — undesirable when latency must be minimized.

3. Solution Development

- Use a vectored interrupt controller: each sensor is wired to a distinct interrupt line; the controller supplies a unique vector number that indexes directly into an Interrupt Vector Table (IVT), so the correct ISR is entered in $O(1)$ time without polling.
- Keep each ISR minimal — the ISR only reads the sensor's data register and stores it into a buffer (top-half); any heavier processing (filtering, computation) is deferred to a low-priority background task (bottom-half), so hardware interrupt time stays short.
- Choose Memory-Mapped I/O: sensor control/status/data registers are mapped into the normal memory address space, so ordinary load/store instructions access them with the same latency as memory operations — no separate instruction decode path, giving the lowest and most predictable latency.
- Assign interrupt priorities so time-critical sensors get higher-priority vectors and can pre-empt lower-priority ISRs if required.

4. Computation / Performance Estimate

Assume 4 sensors, each interrupting every 1 ms, and a well-optimized ISR requiring 20 μs of CPU time per interrupt (read + buffer + return).

Parameter	Value
Number of sensors (N)	4
Interrupt period per sensor	1 ms
ISR execution time per interrupt	20 μs
Total ISR time per 1 ms window	$4 \times 20 \mu\text{s} = 80 \mu\text{s}$
CPU Utilization	$(80 \mu\text{s} / 1000 \mu\text{s}) \times 100 = 8\%$

Result: 8% CPU utilization, which satisfies the <15% constraint with headroom for other system tasks.

5. Interpretation / Justification

Vectored interrupts eliminate device-polling overhead, directly reducing the CPU time spent per interrupt and keeping utilization low even at a 1 kHz interrupt rate. Memory-mapped I/O was selected over I/O-mapped I/O because it avoids dedicated I/O instruction decoding and lets the compiler use flexible addressing modes, which lowers per-access latency — critical for a real-time system responding to sensors every millisecond. The

computed 8% utilization confirms the proposed architecture (vectored interrupts + memory-mapped I/O + short ISR/deferred-processing split) meets all stated constraints.

Question 2: High-Speed NVMe/PCIe I/O Subsystem

Constraints Given

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

1. Problem Understanding

The task is to design a storage I/O subsystem that sustains more than 3 GB/s throughput using the NVMe protocol running over a PCIe physical/link layer, while ensuring the CPU never performs the actual bulk data movement — this must be delegated to a DMA engine, with the CPU informed only via an interrupt when a transfer completes.

2. Constraint Analysis

- Throughput constraint (>3 GB/s): the underlying PCIe link must offer enough raw lane bandwidth; the number of PCIe lanes and generation directly bound achievable throughput.
- Protocol constraint: NVMe (unlike legacy AHCI/SATA) is designed for PCIe-attached flash — it supports up to 64K queues with up to 64K commands per queue, enabling deep parallelism that lets the SSD sustain very high throughput.
- CPU-offload constraint: bulk transfer must bypass the CPU data path entirely, which mandates a DMA engine that moves data directly between the NVMe controller and system RAM.
- Notification constraint: the CPU still needs to know when data is ready — this requires an efficient completion-signalling interrupt mechanism that does not itself become a bottleneck.

3. Solution Development

- Physical link: attach the NVMe SSD controller via PCIe Gen3 x4 lanes (or PCIe Gen4 x2, which delivers comparable bandwidth) directly to the root complex for minimal latency.
- Protocol: use NVMe with paired Submission Queues (SQ) and Completion Queues (CQ) in host memory; the CPU only writes commands into the SQ (a lightweight operation) and never touches the payload data.
- Data movement: an on-controller DMA (bus-master) engine reads/writes payload data directly between the SSD and system RAM using the addresses supplied in each NVMe command — the CPU is completely bypassed for the bulk transfer.
- Completion notification: use MSI-X (Message Signalled Interrupts, extended) so each completion queue can signal the CPU on completion, with interrupt coalescing/aggregation configured so many completions can be reported with far fewer interrupts, avoiding CPU overload.

4. Computation / Performance Estimate

Parameter	Value
-----------	-------

PCIe Gen3 raw rate per lane (8b/10b)	$\approx 0.985 \text{ GB/s}$
Lanes used	x4
Theoretical link bandwidth	$4 \times 0.985 \approx 3.94 \text{ GB/s}$
NVMe/PCIe protocol efficiency (typical)	$\approx 90\%$
Effective achievable throughput	$3.94 \times 0.90 \approx 3.55 \text{ GB/s}$

Result: effective throughput $\approx 3.55 \text{ GB/s}$, which satisfies the $>3 \text{ GB/s}$ requirement with margin. (PCIe Gen4 x4 would provide roughly double this bandwidth if future headroom is needed.)

5. Interpretation / Justification

Running NVMe over PCIe x4 supplies sufficient raw bandwidth ($\approx 3.55 \text{ GB/s}$ effective) to clear the 3 GB/s target. Because the DMA engine performs all payload movement, CPU involvement is limited to submitting lightweight commands and handling a small, coalesced number of completion interrupts — so CPU utilization for the bulk transfer itself is close to 0%. This architecture (NVMe queues + bus-master DMA + MSI-X) is the standard, justified approach for high-throughput, low-CPU-overhead storage I/O.

Question 3: Multi-Device Communication Architecture (USB + Thunderbolt)

Constraints Given

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

1. Problem Understanding

The system connects 6 standard peripherals over USB and 2 high-bandwidth devices over Thunderbolt, sharing a common host. The design must guarantee that no device is starved of interrupt service, must combine hardware interrupts (for immediate device events) with software interrupts (for deferred/background work), and must arbitrate bus access fairly among all attached devices.

2. Constraint Analysis

- Device mix: USB devices are typically lower-bandwidth/latency-tolerant (keyboard, mouse, storage, etc.), while Thunderbolt devices (PCIe-tunnelled) demand high, guaranteed bandwidth and low latency.
- Starvation risk: if Thunderbolt devices are always given strict priority, the 6 USB devices could be indefinitely delayed whenever Thunderbolt traffic is continuous — this must be prevented.
- Fair arbitration: the shared bus/interrupt controller must guarantee every device is serviced within a bounded time, not just the highest-priority ones.
- Interrupt-type constraint: some events (data-ready on high-speed link) need immediate hardware interrupts; other events (status polling, cleanup, logging) can be deferred as software interrupts to reduce hardware ISR load.

3. Solution Development

- USB side: connect the 6 peripherals through a USB host controller using its built-in frame/micro-frame scheduling, which time-slices control, interrupt, bulk, and isochronous transfers so every device gets guaranteed slots within each (micro)frame.
- Thunderbolt side: connect the 2 high-speed devices via Thunderbolt's PCIe-tunnelling with bandwidth reservation, so each gets a guaranteed QoS allocation instead of monopolizing the shared link.
- Priority + aging scheme: implement a priority-based vectored interrupt controller where Thunderbolt devices get a higher base priority (reflecting their real-time needs), but combine this with an aging mechanism — a USB device's effective priority is increased the longer it waits, guaranteeing it is eventually serviced and preventing starvation.
- Hardware vs software interrupts: use hardware interrupts only for urgent, time-critical events (e.g., Thunderbolt data-ready, USB transfer-complete); route routine/background tasks (e.g., status checks,

disconnect/reconnect housekeeping) through software interrupts (traps) serviced at a lower priority, so they never block urgent hardware events.

- Bus arbitration: use a round-robin (or weighted round-robin) bus arbiter so every device gets a bounded maximum wait time, combined with the priority/aging scheme above for interrupt servicing.

4. Computation / Performance Estimate

Device Group	Count	Link	Aggregate Bandwidth Available
USB peripherals	6	USB 3.x (shared host controller)	up to 5–10 Gbps shared
Thunderbolt devices	2	Thunderbolt (PCIe tunnel)	up to 40 Gbps per port

With weighted round-robin arbitration and an aging increment (e.g., +1 priority level per frame missed), the worst-case wait for any USB device is bounded to a fixed number of arbitration cycles — bounded waiting eliminates indefinite starvation even under continuous Thunderbolt traffic.

5. Interpretation / Justification

Separating traffic by link type (USB frame scheduling vs Thunderbolt bandwidth reservation) satisfies each device class's native requirements. The priority + aging interrupt scheme ensures Thunderbolt devices get the low latency they need without permanently starving the 6 USB peripherals, since waiting devices are guaranteed to eventually win priority. Splitting hardware interrupts (urgent) from software interrupts (deferred/background) keeps the critical interrupt path short, improving overall responsiveness while still handling housekeeping work. Together, this satisfies fairness, starvation-avoidance, and mixed hardware/software interrupt requirements.

Question 4: Data Acquisition System (DMA vs Interrupt-Driven I/O)

Constraints Given

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

1. Problem Understanding

A data-acquisition front end (e.g., a high-speed ADC) produces a continuous stream at 500 MB/s that must be moved into system memory with minimal CPU involvement. The design must justify, with numbers, why DMA transfer is chosen over interrupt-driven I/O, and must use memory-mapped I/O to configure the device's control/status registers.

2. Constraint Analysis

- Data rate constraint: 500 MB/s is extremely high; if the CPU had to be interrupted for every small unit of data, the interrupt rate would be unmanageable.
- Minimal-CPU-intervention constraint: the CPU can, at most, set up transfers and respond to occasional completion events — it cannot touch every data word.
- Register-access constraint: device control/status/buffer-address registers must be reachable with simple, fast load/store instructions — memory-mapped I/O satisfies this directly.

3. Solution Development

- Use a DMA controller configured in burst/block mode: the CPU (via memory-mapped registers) programs the DMA engine once with a buffer address and length (e.g., a 64 KB buffer), and the DMA engine then streams data directly from the acquisition device into that memory buffer without CPU involvement.
- The DMA controller raises a single interrupt only when an entire buffer is filled (buffer-complete), not per data item — the CPU's ISR simply hands off the full buffer for processing and reprograms the DMA engine with the next buffer (double-buffering/ping-pong scheme to avoid data loss between buffers).
- All device control/status registers (start, stop, buffer base address, buffer length, status flags) are memory-mapped so the CPU configures them using ordinary load/store instructions.

4. Computation — DMA vs Interrupt-Driven I/O

Approach	Transfer Granularity	Interrupts Needed per Second	Feasible?
Interrupt-driven I/O	4 bytes (32-bit word) per interrupt	$500 \times 10^6 / 4 = 125,000,000 / \text{s}$	No — far exceeds practical interrupt-handling rates ($\sim 10^5$ – $10^6 / \text{s}$)

DMA (64 KB buffers)	65,536 bytes per interrupt	$500 \times 10^6 / 65,536 \approx 7,630 /s$	Yes — well within manageable range
---------------------	----------------------------	---	------------------------------------

CPU load under the DMA approach: assuming an ISR overhead of $\sim 2 \mu s$ per buffer-complete interrupt,

- CPU time used = $7,630 \text{ interrupts/s} \times 2 \mu s = 15.26 \text{ ms}$ of CPU time per second
- CPU utilization $\approx 15.26 \text{ ms} / 1000 \text{ ms} \approx 1.5\%$

By contrast, the interrupt-driven approach would require about $16,384\times$ more interrupts than the DMA approach ($125,000,000 / 7,630 \approx 16,384$), which is not achievable by any practical interrupt controller/CPU combination.

5. Interpretation / Justification

The calculation shows interrupt-driven I/O is completely infeasible at 500 MB/s (it would demand well over 100 million interrupts per second), while a DMA-based, buffered approach reduces this to roughly 7,630 interrupts per second — under 2% CPU utilization. This quantitatively justifies choosing DMA transfer over interrupt-driven I/O for this data-acquisition system. Memory-mapped I/O keeps the (infrequent) control-register access simple and fast, complementing the DMA-based bulk data path.

Question 5: Nested Interrupt Handling Mechanism

Constraints Given

- Support nested interrupts
- Maximum interrupt latency $< 10\ \mu\text{s}$
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

1. Problem Understanding

The system must handle interrupts from devices of differing urgency. It must support nesting (a higher-priority interrupt can pre-empt an in-progress lower-priority ISR), guarantee that any interrupt is serviced within $10\ \mu\text{s}$, use vectored dispatch for high-priority devices, and use software interrupts for lower-urgency background work.

2. Constraint Analysis

- Nesting constraint: without nesting, a high-priority, time-critical interrupt could be forced to wait behind a long-running low-priority ISR, risking large worst-case latency — this must not be allowed.
- Latency constraint ($< 10\ \mu\text{s}$): every stage of the interrupt path — detection, vector fetch, context save, and dispatch — consumes time and must together stay under $10\ \mu\text{s}$.
- Dispatch-method constraint: vectored interrupts are required for high-priority devices so the CPU jumps directly to the correct ISR without any software polling delay.
- Background-task constraint: non-urgent tasks must not be handled through the hardware interrupt path (which would waste priority hardware resources); software interrupts (trap/software-triggered exceptions) are appropriate here.

3. Solution Development

- Implement a priority-based interrupt controller (similar in concept to ARM's NVIC) with multiple programmable priority levels and a hardware vector table, so each high-priority device is dispatched directly to its ISR address.
- Enable interrupt nesting: as soon as a device's priority level is confirmed higher than the currently executing ISR's priority, the controller automatically pre-empts the running ISR, saves its minimal context, and services the higher-priority request immediately; the lower-priority ISR resumes afterward.
- Minimize per-stage overhead: use a small, fixed context-save frame (only critical registers), a hardware-maintained vector table (no software search), and keep ISRs short (defer heavy work, as in Q1/Q4).
- Route background/non-urgent work (e.g., logging, periodic housekeeping) through software interrupts (INT/trap instructions) executed at the lowest priority level, only when no pending hardware interrupt

requires attention — this keeps the hardware interrupt latency path completely unaffected by background work.

4. Computation / Latency Budget

Stage	Estimated Time
Interrupt detection / signal propagation	0.5 μ s
Vector fetch (hardware vector table lookup)	1.0 μ s
Context save (minimal register set)	2.0 μ s
Priority arbitration / pre-emption decision	0.5 μ s
ISR dispatch (jump to handler)	1.0 μ s
Total worst-case latency	$\approx$ 5.0 μ s

Result: $\approx$ 5 μ s worst-case latency, comfortably under the 10 μ s requirement, leaving roughly 5 μ s of margin for synchronization/clock-domain-crossing delays.

Nesting example: if a low-priority ISR (total run time 8 μ s) is 2 μ s into execution when a high-priority interrupt arrives, nesting allows the controller to pre-empt immediately and service the high-priority request within its own $\approx$ 5 μ s budget — rather than forcing it to wait the remaining 6 μ s of the low-priority ISR, which would have violated the 10 μ s requirement.

5. Interpretation / Justification

The hardware-vectored, priority-based design achieves an estimated worst-case latency of about 5 μ s — well inside the 10 μ s limit — because vector fetch removes polling delay and a minimal context-save routine limits pre-emption overhead. Nesting is essential: it guarantees that high-priority devices are never blocked behind long lower-priority ISRs, which a non-nested design could not guarantee. Using software interrupts exclusively for background tasks keeps them out of the time-critical hardware interrupt path entirely, so they cannot threaten the latency budget.

Summary of Design Choices

Q.No	Key I/O Technique	Key Result
1	Vectored interrupts + Memory-mapped I/O	8% CPU utilization (< 15% target)
2	NVMe over PCIe Gen3 x4 + DMA + MSI-X	≈3.55 GB/s effective throughput (> 3 GB/s target)
3	USB frame scheduling + Thunderbolt QoS + priority/aging interrupts	Fair, starvation-free multi-device arbitration
4	Burst-mode DMA with 64 KB buffers + memory-mapped registers	≈1.5% CPU utilization vs. infeasible 125M int/s for interrupt-driven I/O
5	Nested vectored interrupts + software interrupts for background work	≈5 μs worst-case latency (< 10 μs target)